Pure-Python column generation layer

Against the C++-backed layer: the full comparison

Leo Marbehan

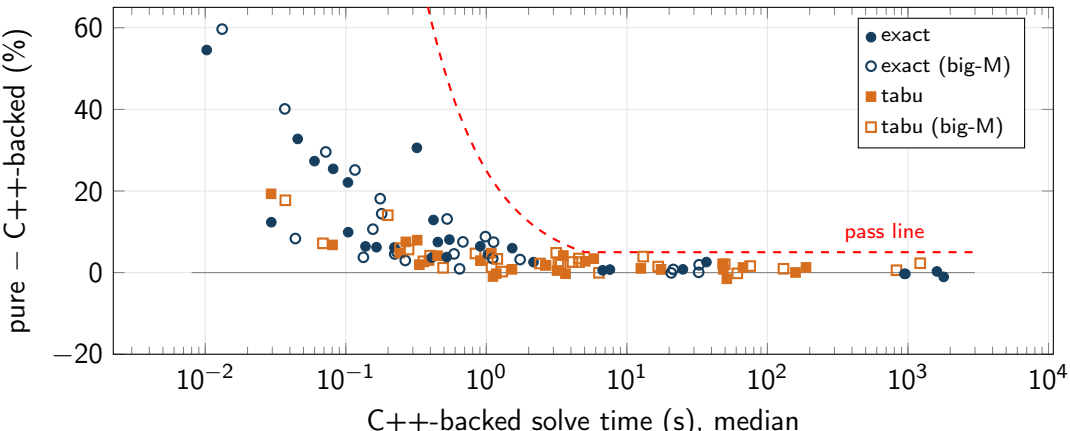
November 8, 2026 (draft)

In one slide

- 108 cases: 14 instances $\times$ {highspy, gurobipy} $\times$ {exact, tabu} $\times$ {single-customer, big-M start}
- 1,204 runs in all (full comparison + overnight re-runs), **0 failed**
- **Same LP value, iterations and columns** in every case: the layers take the same path
- Pure layer: **1–5 ms more per iteration**: visible below 1 s, noise above 10 s
- Rule (D-139): pass when $P - C \leq \max(5\,\% \cdot C,\ 0.25\,\mathrm{s})$

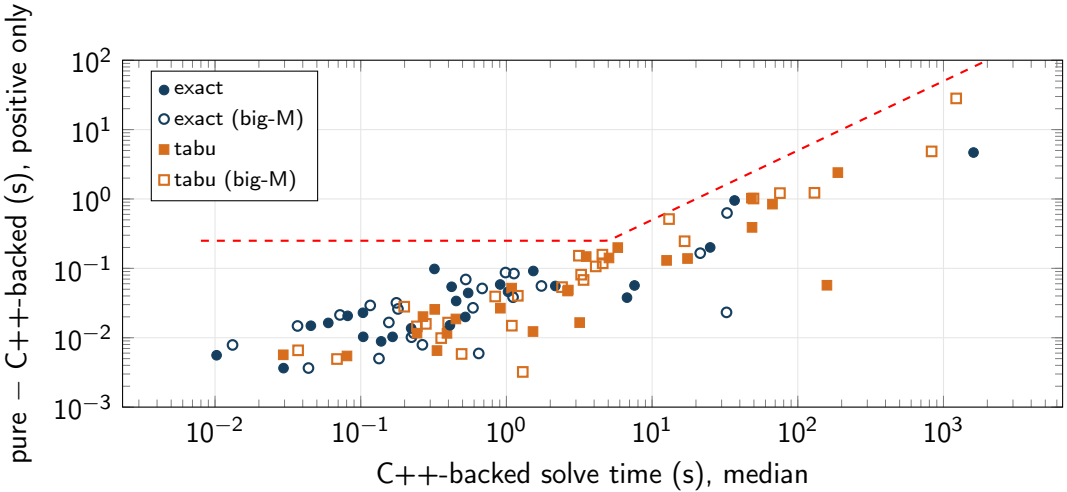
$\Rightarrow$ **108 / 108 pass** (also with a 0.1 s floor) $\Rightarrow$ proposal: **Python only**

Pure layer's extra time, in %, against run length



Every point below the dashed line passes. One point per case; small tier from the quieter overnight re-run.

Pure layer's extra time, in seconds



Log scale: the 10 cases where the pure layer is faster are not shown.

By run length

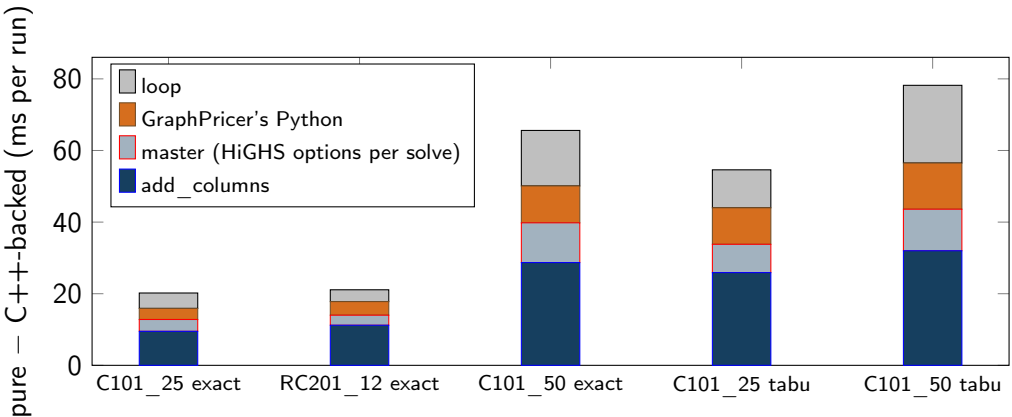
C++-backed run	cases	range (%)	median (%)	largest (s)	pure faster
0–1 s	52	+0.9 to +59.7	+7.5	+0.098	0
1–10 s	30	-0.9 to +7.5	+2.5	+0.199	4
10–100 s	17	-1.5 to +3.9	+1.0	+1.215	3
over 100 s	9	-1.0 to +2.3	+0.3	+28.145	3

- Below 1 s: up to +60 %, but at most +0.1 s
- Over 100 s: -1 to $+2.3$ %: within run-to-run noise (R201 exact: ± 5 % between repeats)

The long cases (C++-backed over 30 s)

instance	master	pricing	start	C++ (s)	pure (s)	diff (s)	diff	repeats
C201	gurobipy	exact	bigm	32.4	32.4	+0.02	+0.1 %	3
C201	highspy	exact	bigm	32.7	33.3	+0.63	+1.9 %	3
C201	highspy	exact	single	36.8	37.8	+0.95	+2.6 %	3
RC201	highspy	tabu	single	48.0	49.1	+1.03	+2.1 %	2
C201	highspy	tabu	single	48.5	48.9	+0.39	+0.8 %	3
C201	highspy	tabu	bigm	49.7	50.7	+1.02	+2.0 %	3
C201	gurobipy	tabu	single	51.5	50.7	-0.78	-1.5 %	3
C201	gurobipy	tabu	bigm	61.1	61.0	-0.11	-0.2 %	3
RC201	gurobipy	tabu	single	67.0	67.9	+0.84	+1.3 %	2
RC201	highspy	tabu	bigm	75.4	76.6	+1.21	+1.6 %	1
RC201	gurobipy	tabu	bigm	130.1	131.4	+1.23	+0.9 %	1
R201	gurobipy	tabu	single	158.5	158.5	+0.06	+0.0 %	2
R201	highspy	tabu	single	188.3	190.7	+2.40	+1.3 %	2
R201	gurobipy	tabu	bigm	829.1	834.0	+4.85	+0.6 %	1
RC201	gurobipy	exact	single	944.3	941.5	-2.81	-0.3 %	2
RC201	highspy	exact	single	958.6	955.7	-2.91	-0.3 %	2
R201	highspy	tabu	bigm	1222.4	1250.5	+28.14	+2.3 %	1
R201	gurobipy	exact	single	1608.1	1612.8	+4.67	+0.3 %	2
R201	highspy	exact	single	1796.9	1778.1	-18.86	-1.0 %	2

Where the extra milliseconds go (gap probe, F-47)



18–67 iterations: about 1–2 ms per iteration, plus 2–5 ms once. The pricing search is the same C++ call.

The master-heavy case: C201 (190 iterations, 38,000 columns)

	exact	tabu
C++-backed (s)	35.58	46.04
pure (s)	36.52	46.79
difference, total (ms)	+942 (+2.6 %)	+753 (+1.6 %)
master LP solves	+230	+309
pricing slice	+615	+380
of which `add_columns` (pure, whole)	342	268
of which GraphPricer's Python	108	101
loop rest	+62	+64

Probe, 3 repeats. Full matrix on C201: -1.5 to $+2.6$ %, all pass.

Decisions to take

1. **D-139**: the rule, floor 0.25 s (outcome identical with 0.1 s)
2. **Q-41 / D-141**: Python only
   - gain: one implementation, no bindings layer for the loop
   - cost: C++ users lose the column generation layer (supersedes D-02, D-08, D-15, D-37)
   - stack: drafts #61, #69, #72, #76, Python PRs 5–6, #80 reworked or closed
3. **D-140**: no C++ helper for `add_columns` (saves $\leq 0.2\,\%$ on C201)

Caveats: long tier 1–2 repeats; desktop not fully idle; exact pricing from big-M on R201/RC201 not run (pricing $> 99\,\%$).